

OSI4IOT Platform

Daniel Di Capua

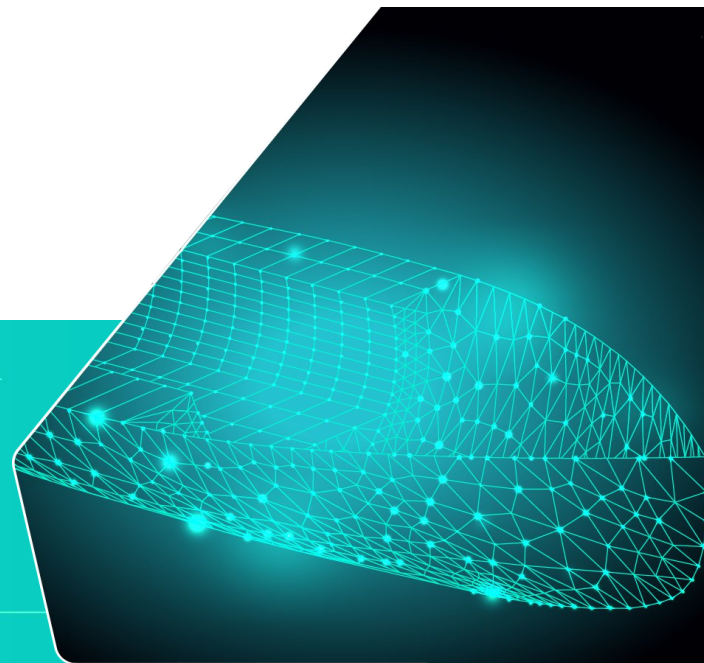
Profesor Agregado Dpto. Resistencia de Materiales y Estructuras a la Ingeniería de la UPC

Centro Internacional de Métodos Numéricos en Ingeniería (CIMNE)

Colaborador empresa Compass Ingeniería y Sistemas, SA



This project has received funding from European Union's Horizon 2020 research and innovation programme under grant agreement n° 101006860

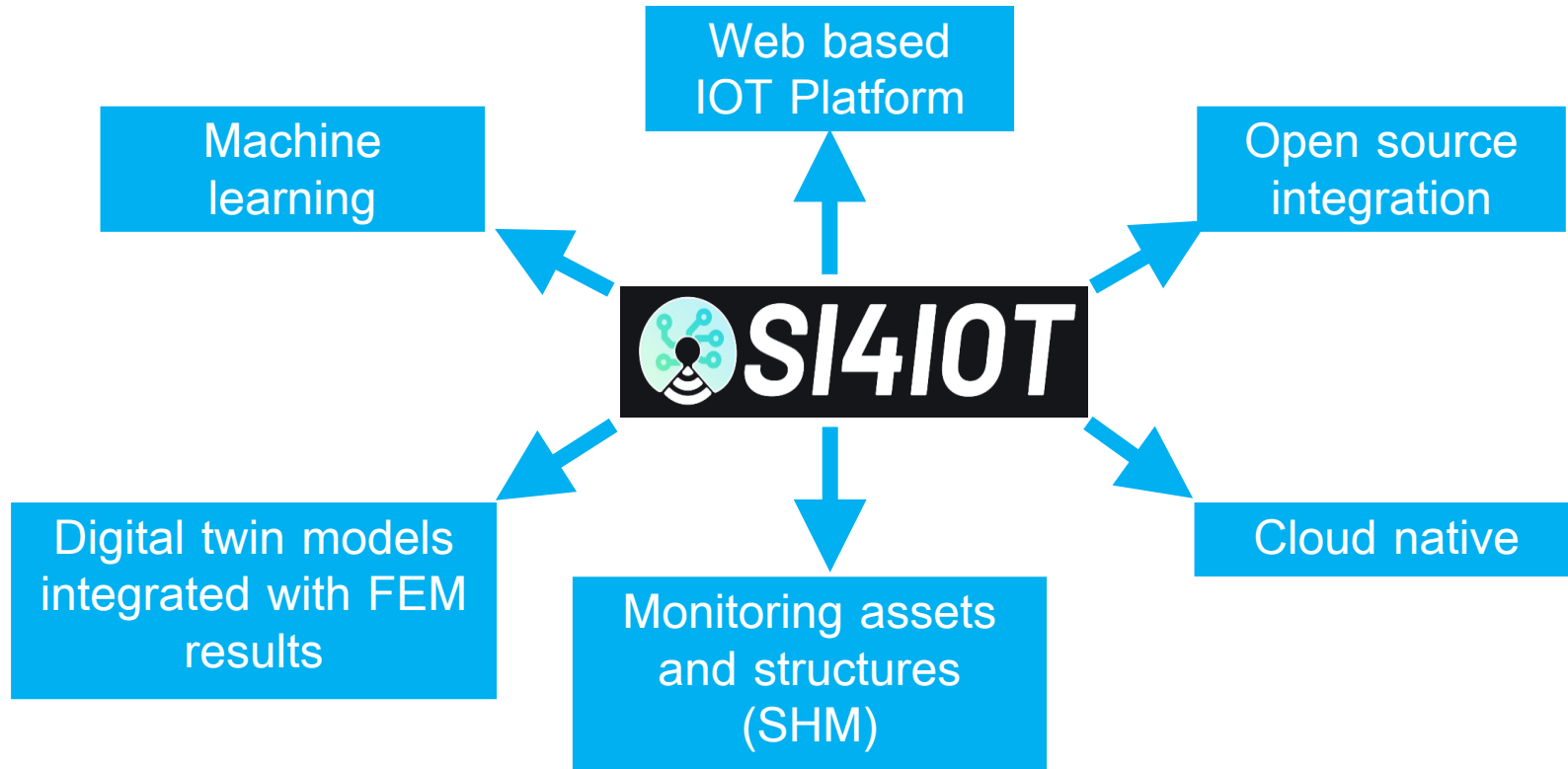




Open Source Integration For Internet Of Things

<https://github.com/osi4iot/osi4iot>

What is OSI4IOT?





Well known open source
packages

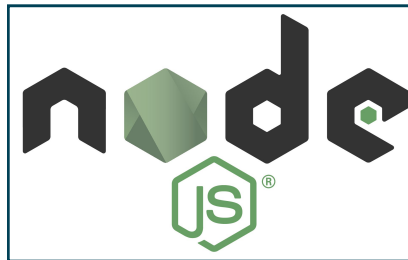


Custom code
(also open source)

Well known open source packages



Custom code



OSI4IoT

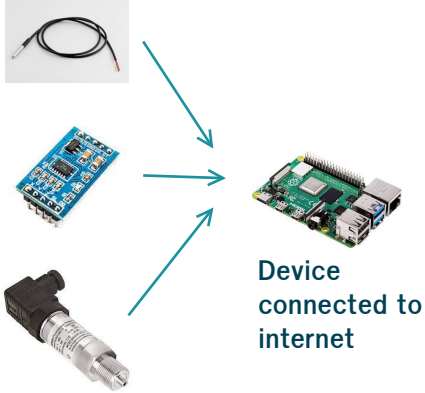
Open Source Integration For Internet Of Things



Sensors

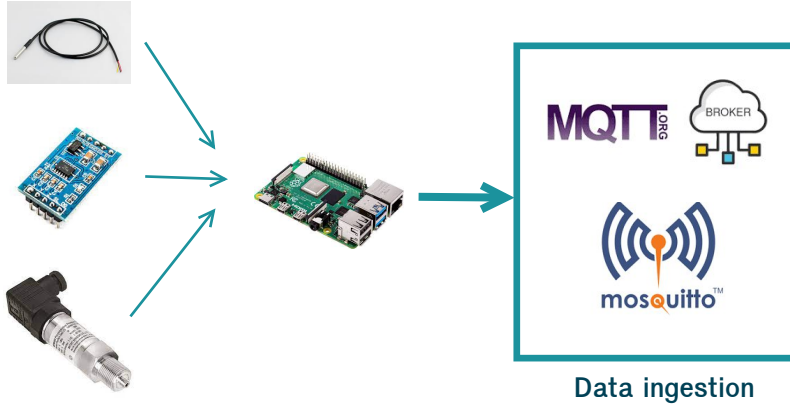
OSI4IoT

Open Source Integration For Internet Of Things



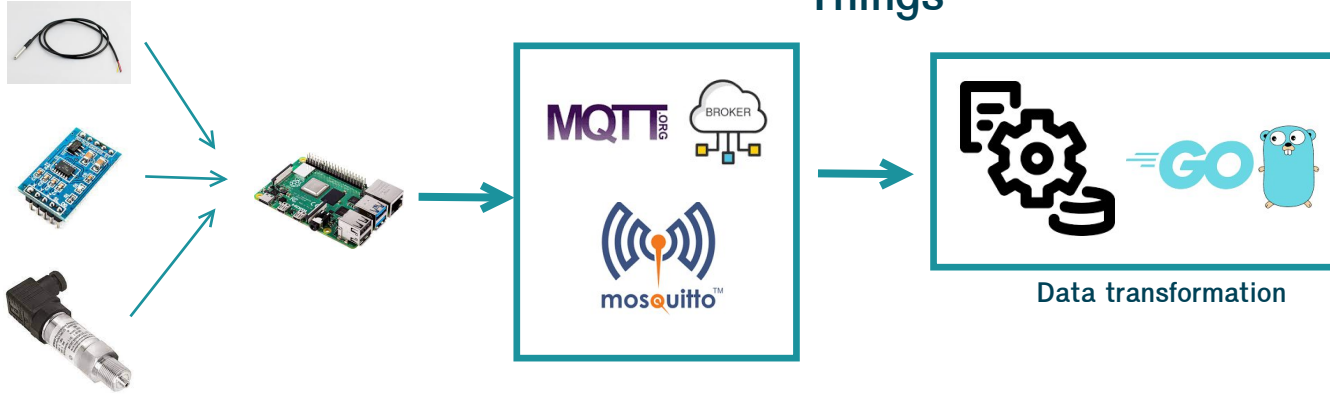
OSI4IoT

Open Source Integration For Internet Of Things



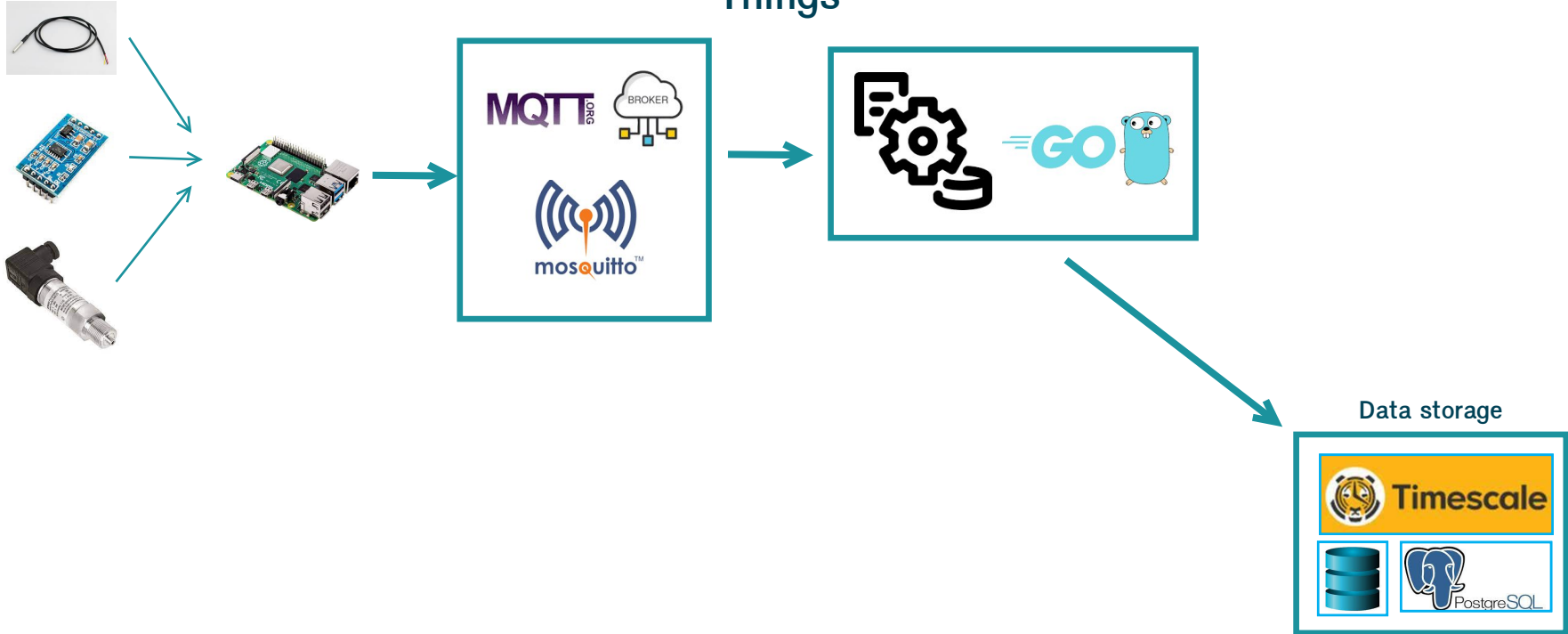
OSI4IoT

Open Source Integration For Internet Of Things



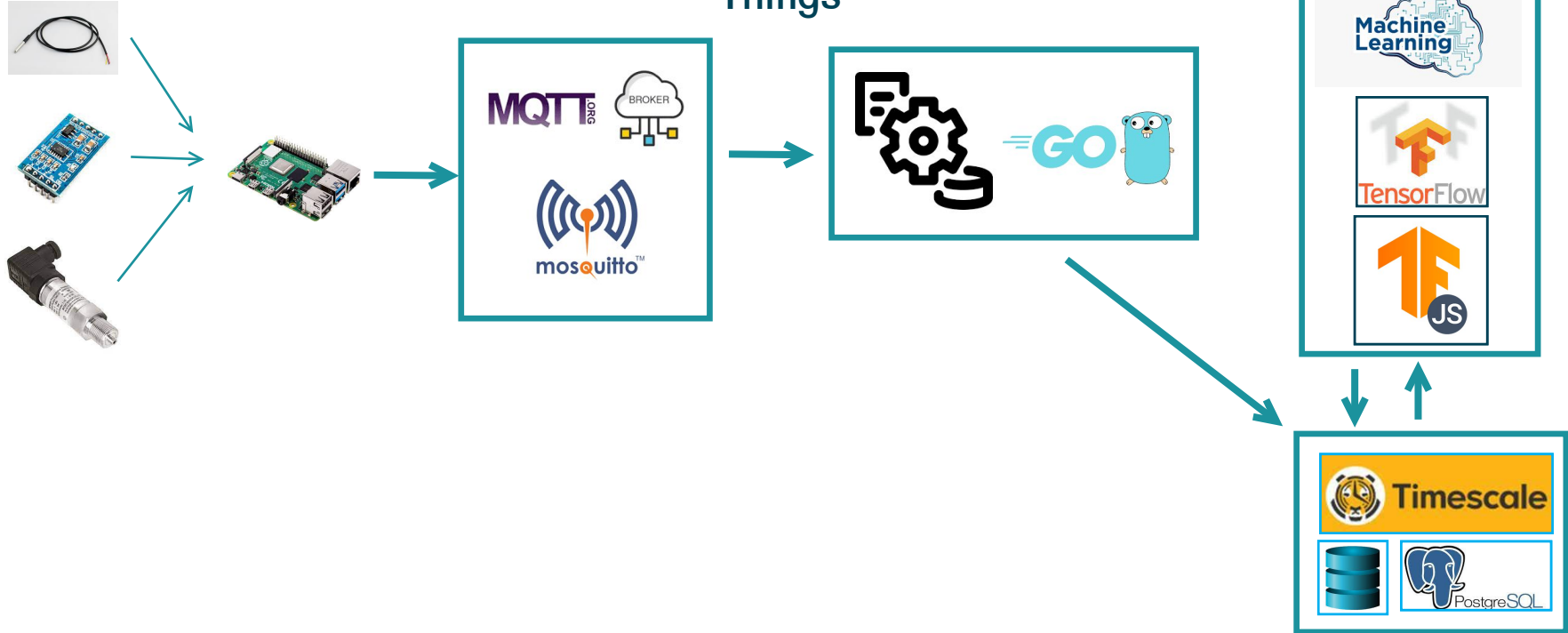
OSI4IoT

Open Source Integration For Internet Of Things



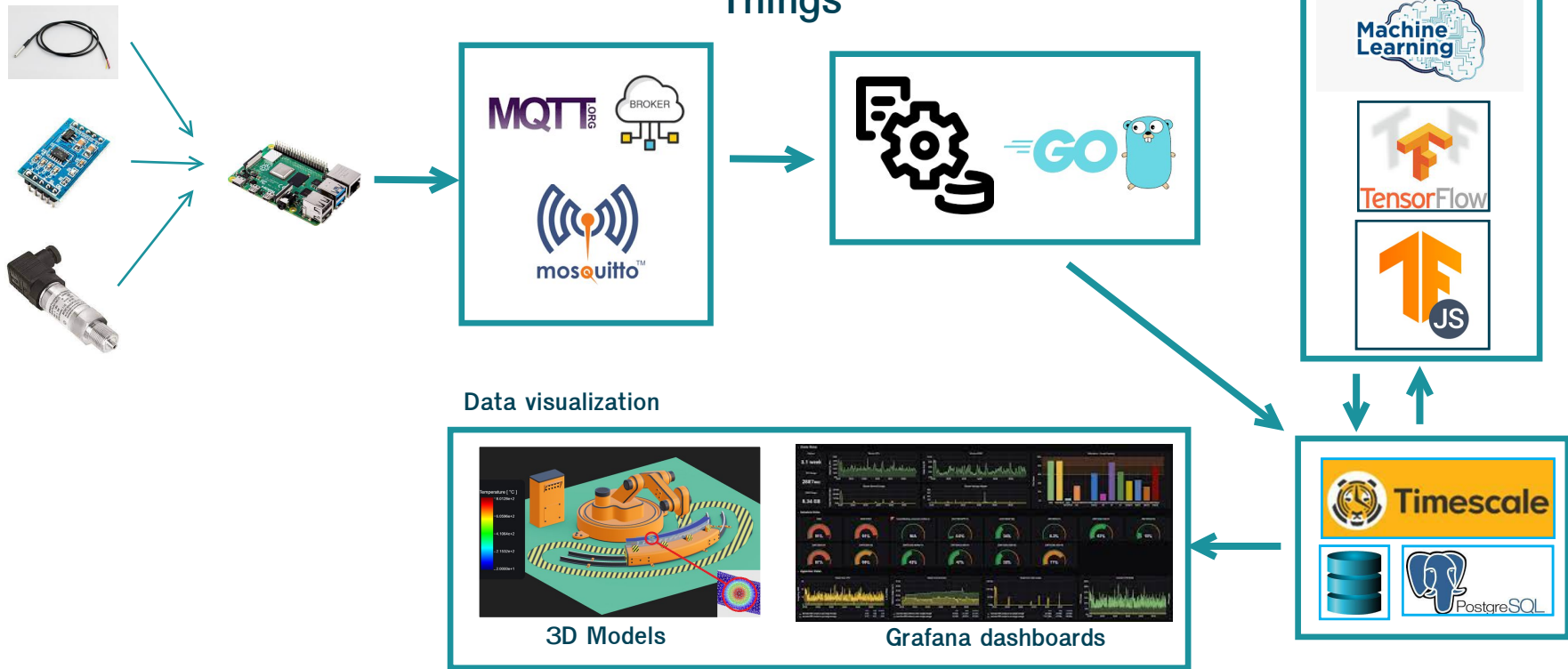
OSI4IoT

Open Source Integration For Internet Of Things



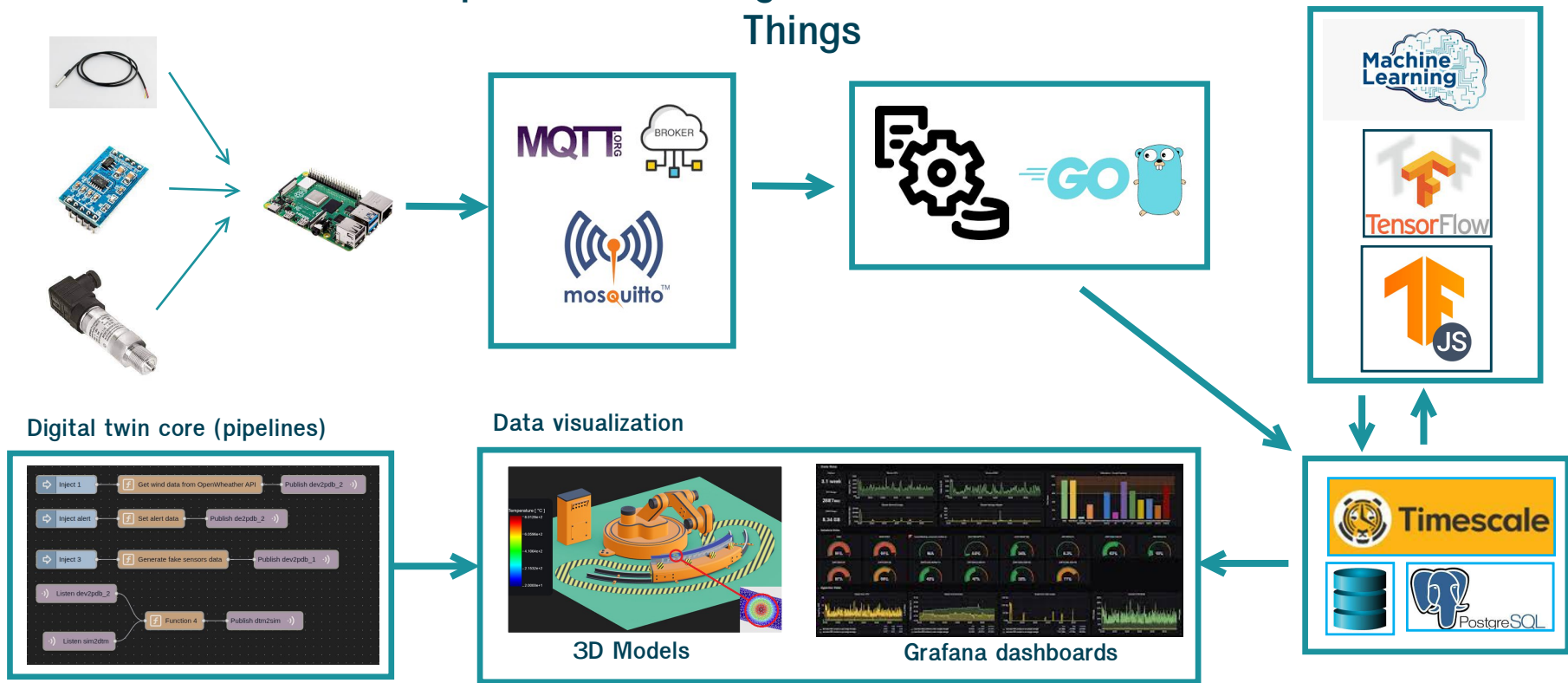
OSI4IoT

Open Source Integration For Internet Of Things



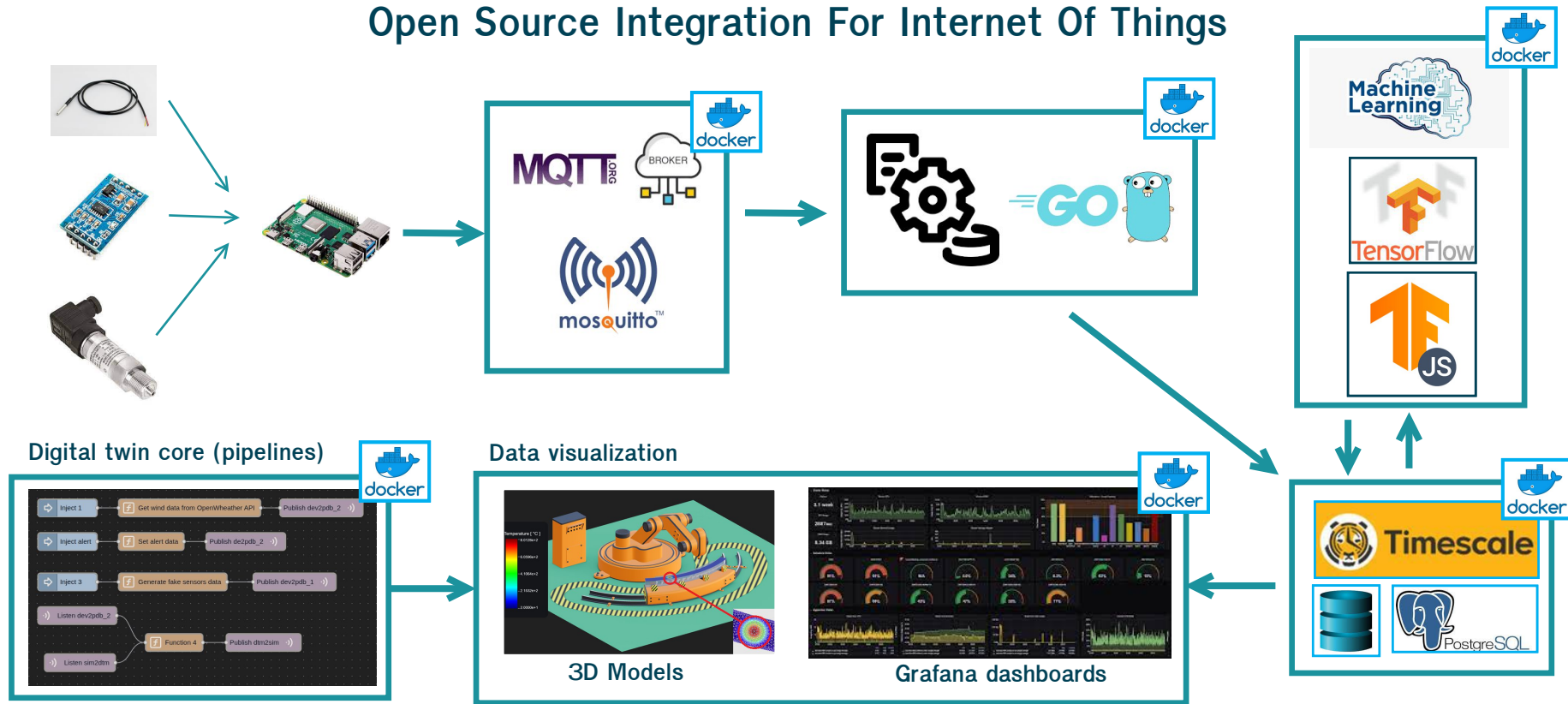
OSI4IoT

Open Source Integration For Internet Of Things

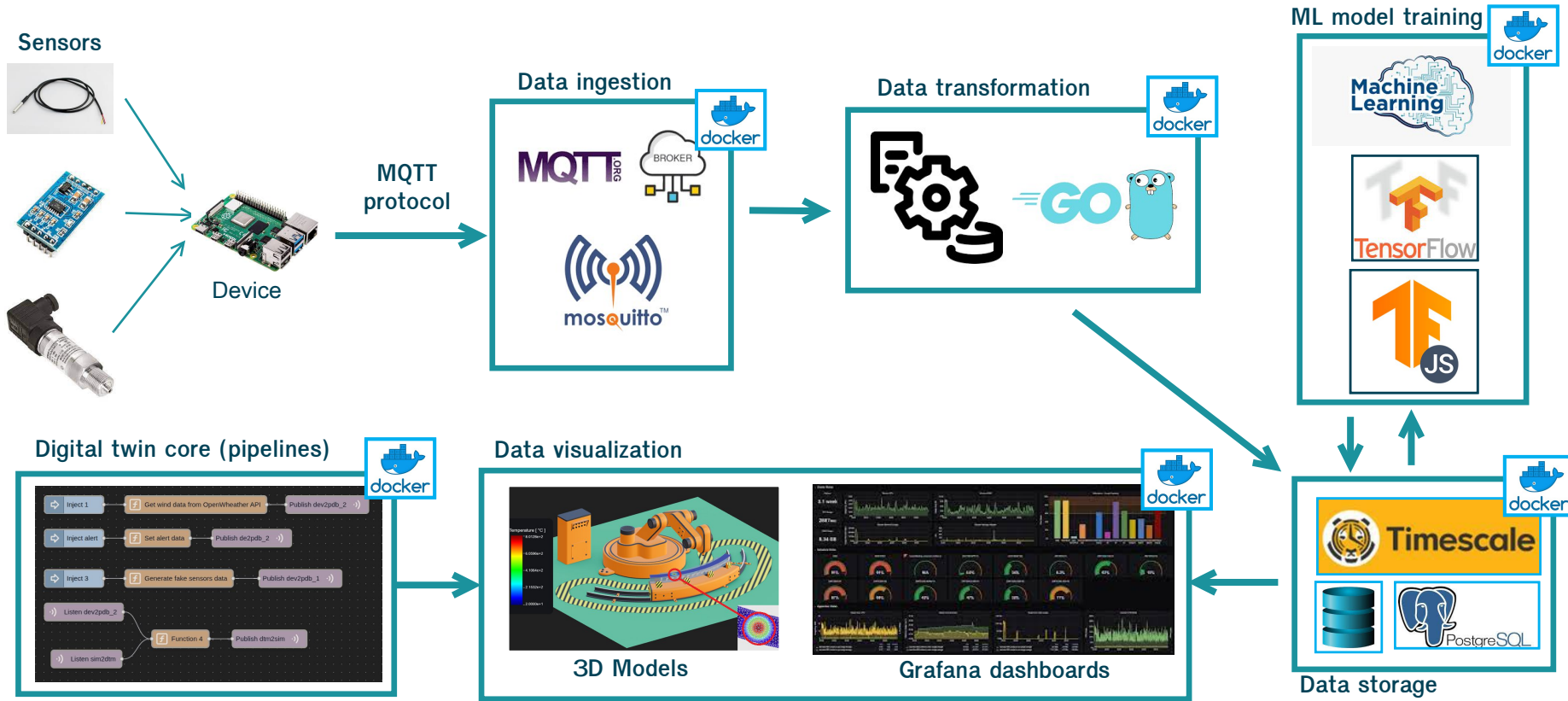


OSI4IoT

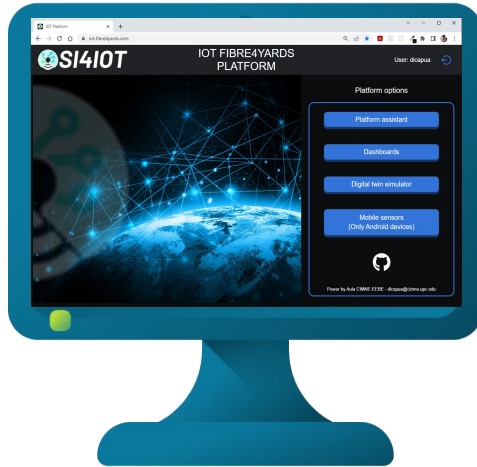
Open Source Integration For Internet Of Things



OSI4IoT Architecture



WEB-BASED IOT PLATFORM



Web user
interface

Features



User registration



Create organizations



Create private groups



Assets registration



Sensors registration



Dashboards



Digital twins



Machine learning models

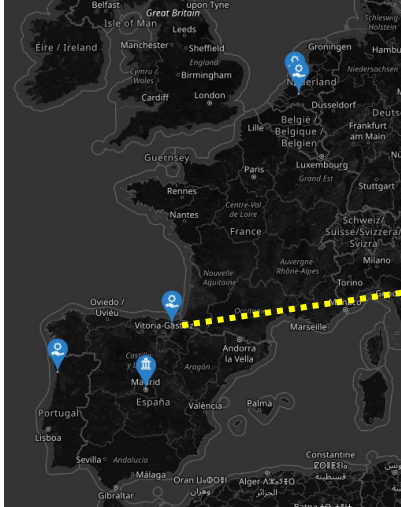


3D models



FEM results integration

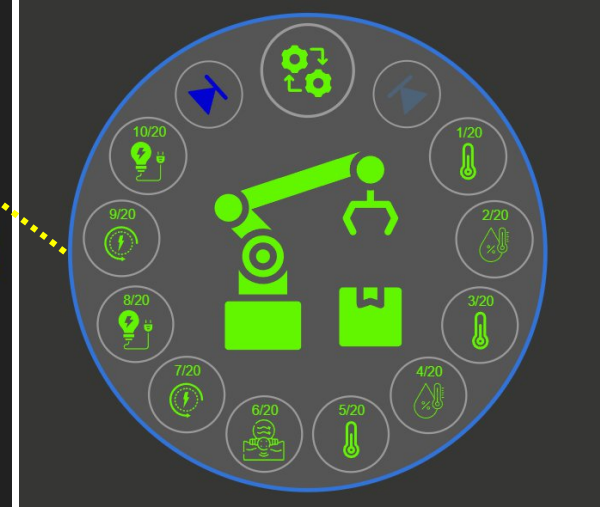
Org assets and sensors geolocation



Organization geolocation

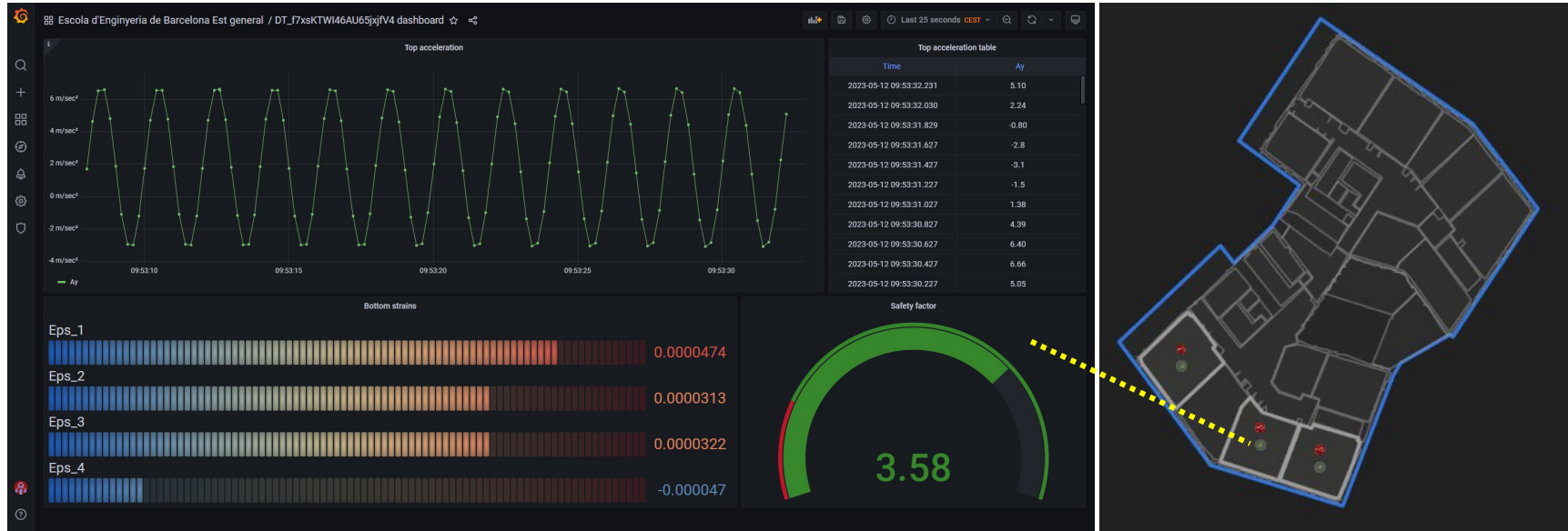


Organization building



Asset, sensors and digital twin

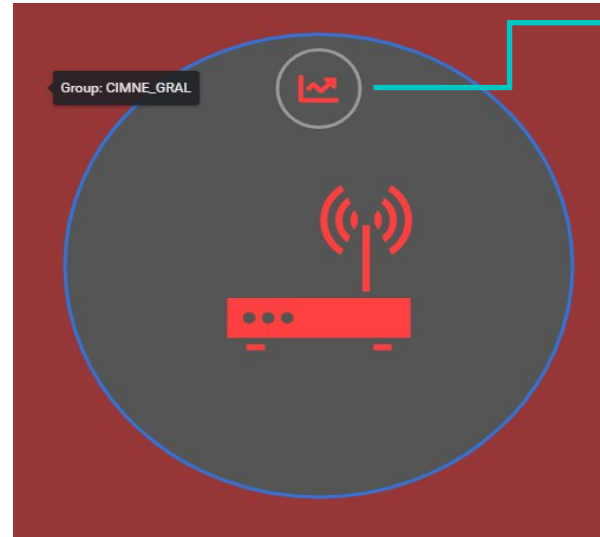
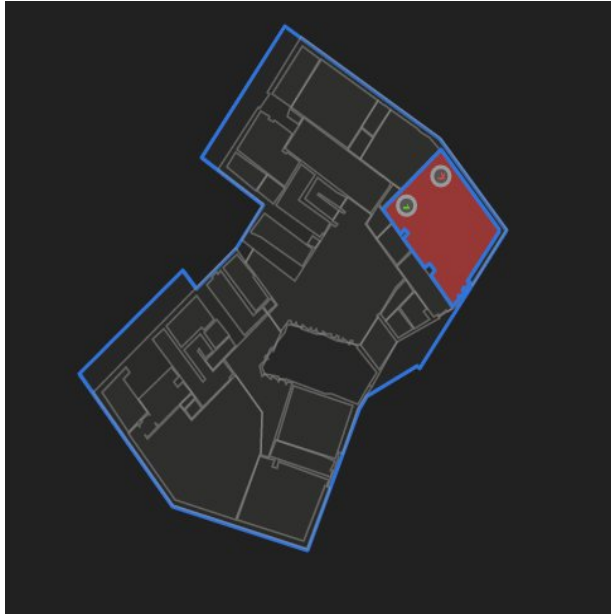
Grafana dashboards:



□ Digital twin model represented by a Grafana dashboard.

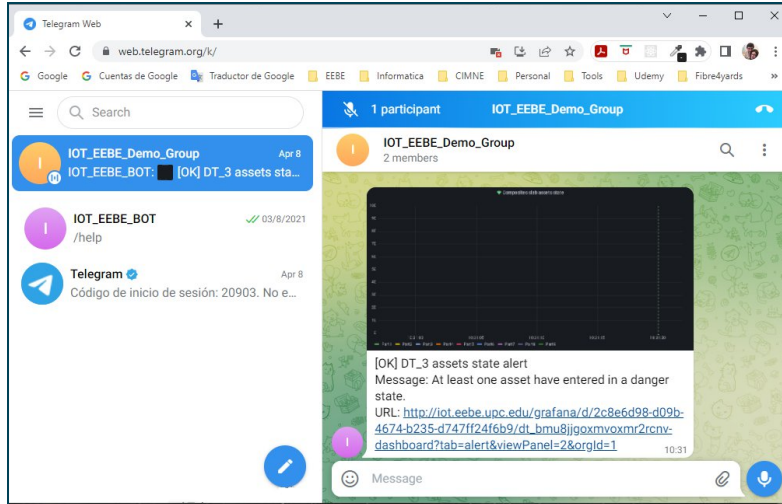
Alerts geolocation

- When same alert is triggered the platform detect in real time the geolocation of the group and device with problems.

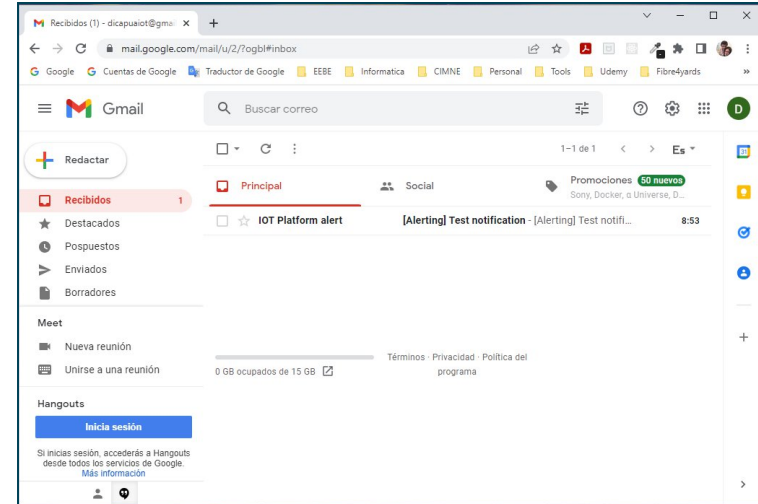


Link to the
corresponding
Grafana dashboard

Alert notification channels:

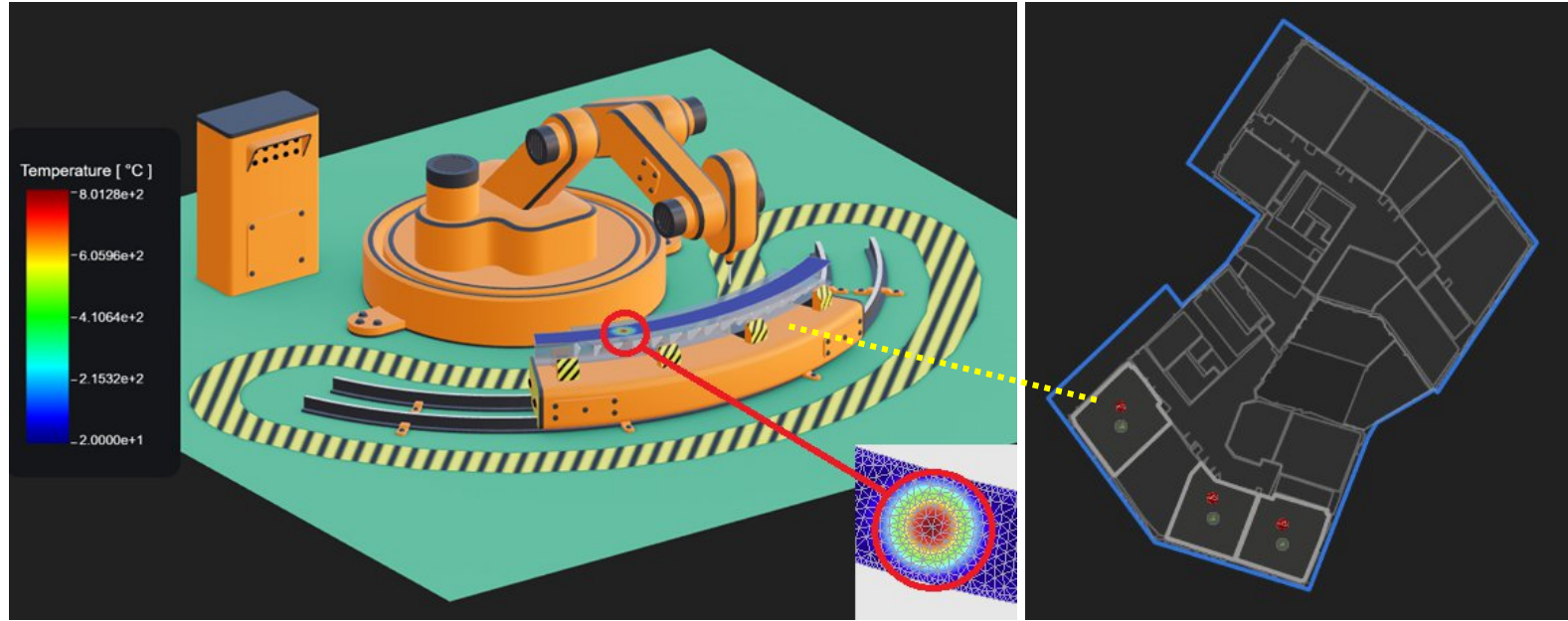


Telegram



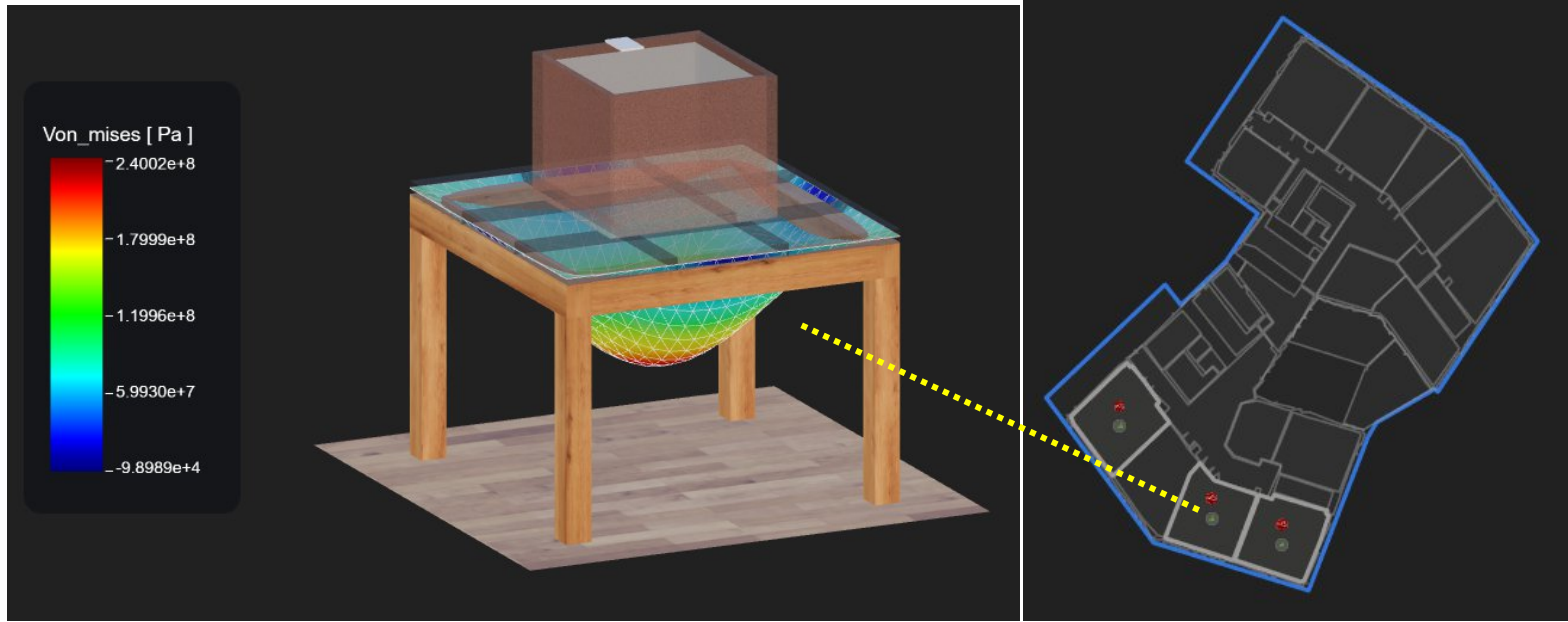
Email

3D models:



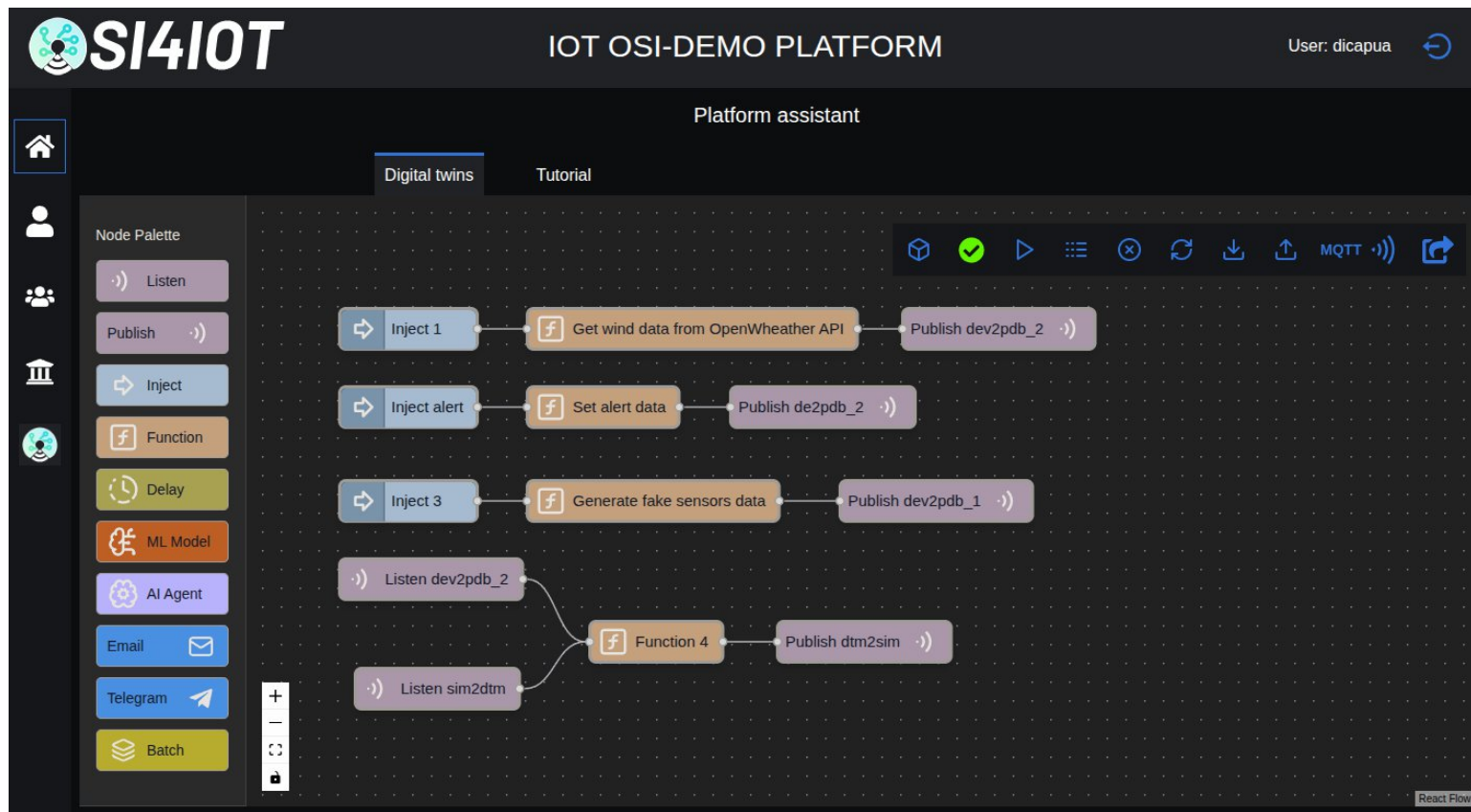
- Digital twin model represented by gltf 3D model and FEM simulations.

3D models integrated with FEM results:

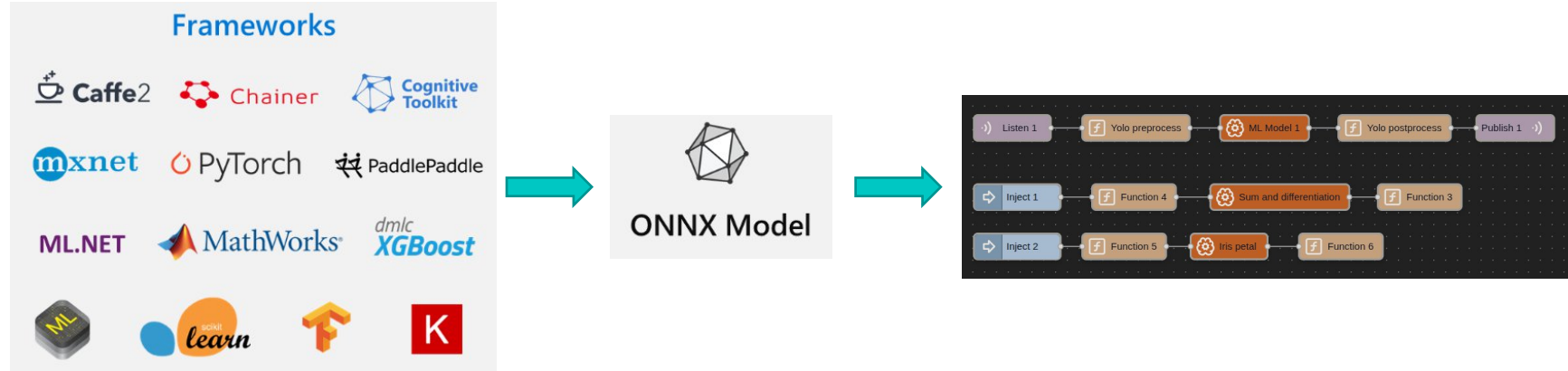


- Digital twin model represented by gltf 3D model and FEM simulations.

Custom pipelines



Machine learning models



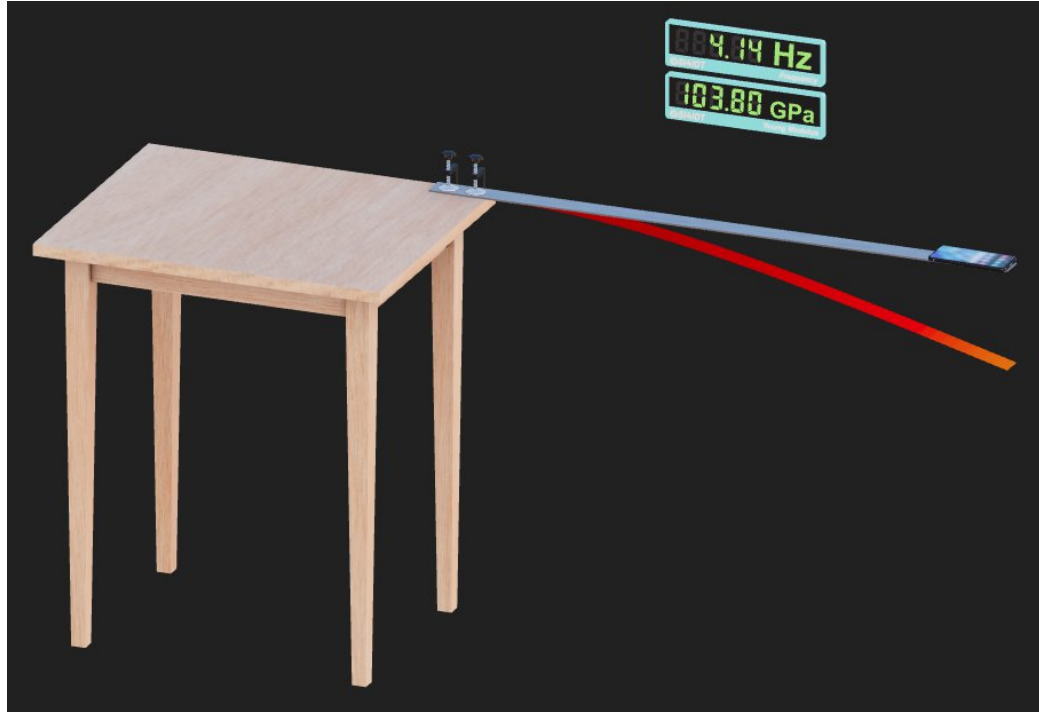
□ Development process of MLM in OSI4IoT platform

Application case

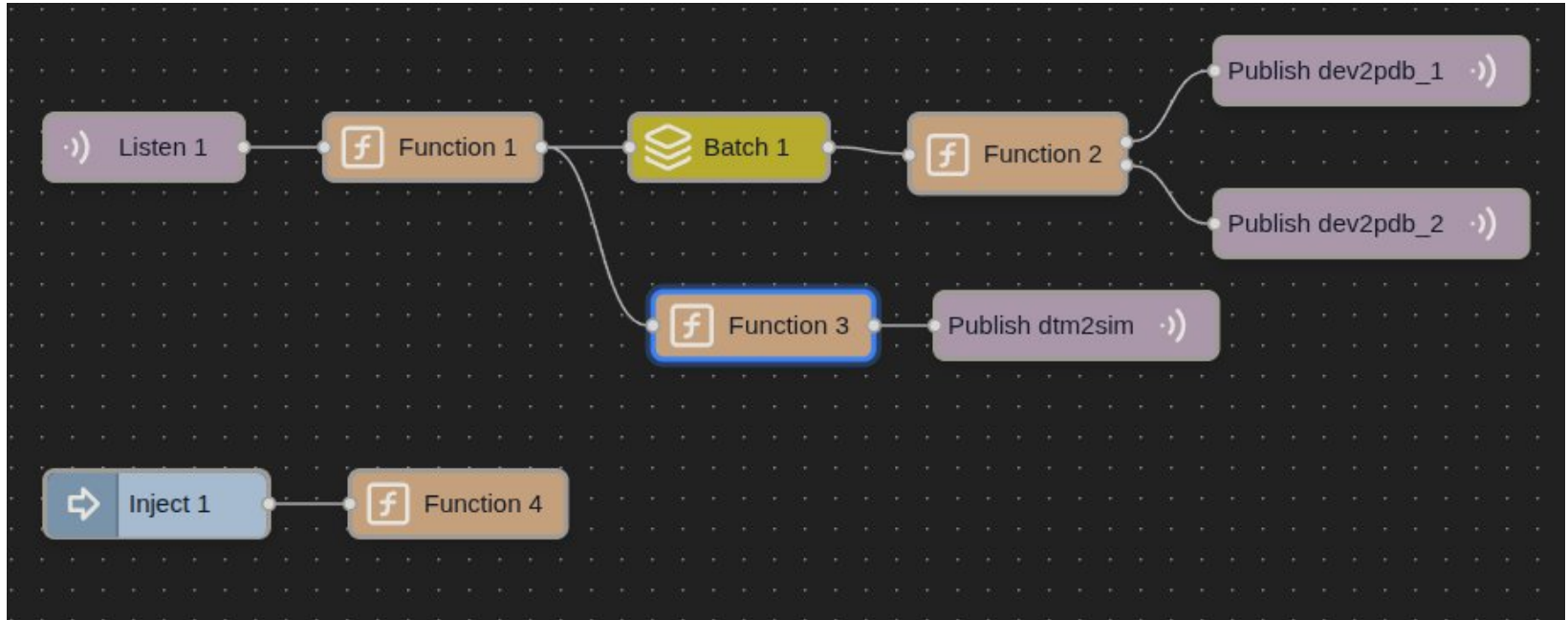
Aluminium cantilever beam



Digital twin



Pipeline



Node Function 1

```
1 function process(msg) {  
2   const go = Go();  
3   const { log, time } = go.All();  
4  
5   const timeStr = msg.payload.timestamp;  
6   const timestamp = time.Parse(time.RFC3339,timeStr).UnixMilli();  
7   const mobile_motion = msg.payload.mobile_motion;  
8   const [ax, ay, az] = mobile_motion.slice(0,3);  
9   const [qx, qy, qz, qw] = mobile_motion.slice(3,7);  
10  
11   const v3 = 1.0 -2.0*(qx*qx+qy*qy);  
12   const alpha = Math.acos(v3);  
13  
14   msg.payload = {  
15     alpha,  
16     acceleration: az,  
17     timestamp  
18   }  
19  
20   return msg;  
21 }
```

Node Function 2

```
1 function process(msg) {
2   const go = Go();
3   const { log, time, dsp } = go.All();
4   const batch = msg.payload.batch;
5
6   const fs = 25.0;
7   let times = [];
8   let values = [];
9   for (i=0; i < batch.length; i++) {
10     const item = batch[i];
11     times.push(item.timestamp);
12     values.push(item.acceleration)
13   }
14   [times, values] = dsp.SortTimeSeriesData(times, values);
15   dsp.DetrendInPlace(values);
16
17   const welchConfig = dsp.GetDefaultWelchConfig(fs);
18   const [freq, psd, df] = dsp.WelchPSD(values, welchConfig);
19
20   // Generate Power Spectral Density graph
21   const psdGraph = [];
22   for(let i=0; i<freq.length; i++) {
23     const psdDb = 10.0 * Math.log10(psd[i]+1e-12);
24     const point = {
25       freq: freq[i],
26       psd: psdDb
27     }
28     psdGraph.push(point)
29   }
30 }
```

```
31 // Calculate Peaks
32 const peaks = dsp.FindWelchPeaks(freq, psd, df, welchConfig);
33 const f1_exp = peaks[0].Freq;
34
35
36 const f1_num = 3.52;
37 const young_num = 75.0;
38 const young_modulus = young_num*Math.pow(f1_exp/f1_num, 2.0);
39 log.Infof("f1=%fHz, E=%fGPa", f1_exp, young_modulus);
40
41 const msg1 = {
42   payload: {
43     frequency: f1_exp,
44     young_modulus
45   }
46 }
47
48 const msg2 = {
49   payload: {
50     timestamp: (new Date()).toJSON(),
51     psdGraph
52   }
53 };
54
55 return [msg1, msg2];
56 }
```



Thank you !

